

DOTS: Exactly Removable Continual Learning for Frozen Language Models via Hyperdimensional Sparse Memory

Sohan Merugu

Abstract—A language model that keeps learning after deployment must also be able to forget on request. We present DOTS, a retrofit that attaches a 262,144-slot sparse memory to a frozen pretrained transformer and records every learned unit of knowledge, a *Dot*, in a ledger. Memory addresses are fixed random hypervectors applied to whitened hidden states, so attaching the memory leaves the model’s outputs bit-identical and a learned item is never orphaned by address drift. Each Dot writes only value rows that ordinary text rarely reads and that no earlier Dot owns, using a per-Dot optimizer under deterministic kernels. Because every Dot logs the slots it read and wrote, removing a Dot reduces to restoring rows and replaying only the Dots it could have influenced. The result is bitwise identical to a model that never learned the removed Dot. On Qwen2.5-0.5B learning 1,000 TriviaQA facts it did not know, in ten sequential Dots, DOTS recalls 96.1% of the facts (73.5% under a prompt context never seen in training), keeps 86.3% of facts the model already knew, and changes held-out perplexity by 1.6%. The best full fine-tuning run keeps 51.3% of known facts and the best LoRA run 33.0%. We verify bit-exact removal and reset on both CPU and GPU, and we report the main cost: in our setting every removal replays all later Dots.

Index Terms—Continual learning, machine unlearning, sparse memory, large language models, catastrophic forgetting, hyperdimensional computing.

I. INTRODUCTION

Deployed language models are frozen. A model that could keep learning from the documents its user chooses, on the user’s own machine, would need two properties that pull against each other. It must write new knowledge into its parameters without erasing what it already knows, and it must be able to take any piece of that knowledge back out. Dense weight updates fail the first property [1]–[3]. Approximate unlearning fails the second: knowledge suppressed by gradient-based unlearning returns after 4-bit quantization, from roughly 21% to 83% retained [4].

Sparse memory layers point toward a way to satisfy both. Product-key memories add millions of key-value slots of which each token reads only a few [5], [6]. Updating only the slots that new data uses more than background data does strongly reduces forgetting [7]–[9]. Exact removal, however, needs more than sparsity. Every later update was computed in the presence of earlier ones, so subtracting a logged parameter delta does not recover the state a model would have had without that data. Exact deletion in the literature comes from isolation by design and deterministic replay [10]–[12].

DOTS combines these lines of work into one mechanism. Its contributions are:

- 1) **A training-free retrofit.** A sparse memory is attached in parallel to one MLP block of a frozen model. Addressing uses a fixed random projection of whitened hidden states and fixed random bipolar sub-keys, so nothing in the address path is trained and attaching the memory changes no output bit (§III-A).
- 2) **A write rule for low interference.** Each Dot selects slots by TF-IDF against background usage, never writes slots that ordinary text reads heavily or that earlier Dots own, and trains only those rows (§III-B).
- 3) **A ledger with exact removal.** Dots log read and write sets. We state the conditions under which dropping a Dot and replaying its taint closure is bitwise equal to never having learned it, and verify this on CPU and on GPU (§III-C, §V-A).
- 4) **An empirical comparison** on Qwen2.5-0.5B against tuned LoRA and full fine-tuning, including recall under an unseen prompt context and the removal cost (§V).

II. RELATED WORK

a) *Sparse memory layers.*: Product-key memory [5] and its scaled versions [6] replace or augment feed-forward blocks with large key-value tables. Sparse memory finetuning [7] updates only the value rows most specific to new data relative to background batches. On a 1.3B model learning 1,000 facts it reduced the NaturalQuestions F1 drop from 89% (full fine-tuning) and 71% (LoRA) to 11%. Retrofits into Qwen2.5-0.5B followed [8], [9]. Hierarchical memory banks separate long-tail knowledge from a small core model and allow memory blocks to be deleted [13]. DOTS differs in that its memory is added to an unchanged model without any further training, its addresses are fixed rather than learned, and every write is logged so that removal is exact rather than approximate.

b) *Test-time training and fast weights.*: Fast-weight programmers update a matrix state with each token [14]. Gated delta rules [15], test-time training layers [16], Titans [17], and nested learning [18] treat parts of the network as memories trained during inference. In-place test-time training updates a pretrained model’s own MLP projections [19], and fast-weight product-key memory applies such updates to sparse slots [20]. These methods target in-context adaptation and reset their state at document or request boundaries. DOTS targets persistent knowledge and its reversal.

c) *Knowledge editing.*: Locate-and-edit methods change MLP weights from (subject, relation, object) triples [21]–[23]. Codebook editors store edits in separate key-value entries [24], [25]. Larimar writes to an episodic memory in closed form and removes an entry with the same update [26]. MemoryLLM keeps a self-updating latent memory pool [27]. DOTS learns from ordinary question-answer text rather than triples and keeps its writes addressable for removal.

d) *Continual learning and forgetting.*: Catastrophic interference [1] is commonly reduced by regularizing important weights [2] or replaying earlier data [3]. LoRA [28] learns less and also forgets less than full fine-tuning [29]. Long training sequences also erode plasticity [30]. Dynamic evaluation finds that resetting online updates at document boundaries works best [31], which is evidence against naive always-on full-weight learning.

e) *Machine unlearning.*: Approximate methods such as negative preference optimization [32] suppress outputs but can be undone [4]. Exact methods isolate data at training time: SISA retrain only the affected shard [10], SIFT-Masks merges and unmerges sign-constrained finetunes [12], and deterministic replay reproduces deletion counterfactuals bit for bit at 1–2.8B scale [11]. Gradient routing and selective gradient masking localize a capability so it can be zeroed [33], [34]. FlexOlmo and NULLs isolate data sources architecturally [35], [36]. DOTS applies isolation and replay at the granularity of memory rows, with a dependency analysis that decides which Dots must be replayed.

f) *Hyperdimensional computing.*: Vector-symbolic architectures compute with quasi-orthogonal random high-dimensional vectors [37], [38]. Sparse distributed memory [39] stores data at many hard locations addressed by similarity, and attention approximates it [40]. The capacity of a single superposition vector grows only linearly with dimension [41], which motivates storing knowledge across many slots instead. Random sign projections preserve angles [42], and open tooling for these operations exists [43]. We use these ideas only for addressing: fixed random sub-keys and a fixed random projection.

III. METHOD

A. Retrofitting a sparse memory

Let $x \in \mathbb{R}^d$ be the input to the MLP of decoder layer ℓ (the output of its post-attention normalization). DOTS replaces the block’s MLP output $\text{MLP}(x)$ with $\text{MLP}(x) + M(x)$, where M is a sparse memory with $N = S^2$ slots and value table $V \in \mathbb{R}^{N \times d}$.

a) *Addressing.*: The input is whitened with background statistics, $z = (x - \mu) \oslash \sigma$, and projected by a fixed Gaussian matrix $P \in \mathbb{R}^{d \times 2k}$ with entries $\mathcal{N}(0, 1/d)$. The result $q = zP$ is split into halves $q_1, q_2 \in \mathbb{R}^k$. Each half is matched against S fixed random bipolar sub-keys $K_1, K_2 \in \{-1, +1\}^{S \times k}$, with scores $s_j = q_j K_j^\top / \sqrt{k}$. As in product-key memory [5], the top- t candidates from each half are combined into t^2 slot candidates with score $s_1[a] + s_2[b]$ and slot index $aS + b$. The top- t of these are read with weights $w = \text{softmax}(\text{score}/\tau)$:

$$M(x) = \sum_{i \in \text{top-}t(x)} w_i V_i. \quad (1)$$

P , K_1 and K_2 are drawn once from a seeded generator and never trained. A slot’s address therefore depends only on the frozen model and the input, and does not drift as learning proceeds.

b) *Function preservation.*: V starts at zero, so $\text{MLP}(x) + M(x) = \text{MLP}(x)$ exactly and the retrofitted model’s outputs are bit-identical to the original’s. We verify this on every run.

c) *Calibration.*: Two background statistics are computed with forward passes only. The first is the whitening mean and standard deviation (μ, σ) . The second is, for every slot, its document frequency df_i (how many background sequences read it) and its read mass m_i (the sum of read weights). The background mixes plain text with text in the format the model will be queried in (§IV). This mix matters: with plain text only, slots read by the question-answer format itself look rare and are selected for writing, which damages unrelated questions (Table III).

B. Writing a Dot

A Dot is one batch of training examples from one source, here 100 question-answer facts. Learning a Dot proceeds as follows (Algorithm 1).

- 1) **Cache.** Run layers $< \ell$ once and cache their output. Layers below the memory never change, so training needs forward and backward passes only through layers $\geq \ell$.
- 2) **Read set.** Record every slot the Dot’s tokens read, R , and per-slot read counts c_i .
- 3) **Select.** Score each read slot by $c_i \log \frac{D+1}{\text{df}_i+1}$, where D is the number of background sequences. Exclude slots whose read mass m_i lies above the 90th percentile of all slots (*protection*) and slots written by earlier Dots (*ownership*). Keep the top B slots as the write set W .
- 4) **Train.** Optimize only the rows V_W with a fresh Adam optimizer [44] on the next-token loss over answer tokens. Minibatch order comes from a seed derived from a hash of the Dot’s data.
- 5) **Log.** Store the delta $\Delta = V_W^{\text{after}} - V_W^{\text{before}}$ together with W , R , the seed, the source, and the training examples.

All other parameters, including the address path, stay frozen. Training runs with PyTorch’s deterministic algorithms enabled [45] and a cross-entropy computed from log-softmax and one-hot targets, which avoids atomic additions on GPU. Without deterministic algorithms, two CPU runs of the same Dot differed by up to 1.1×10^{-4} in the learned rows, enough to break bit-exact replay.

C. The ledger: reset and exact removal

The canonical memory state is $V = V_0 + \sum_j \Delta_j$, summed in Dot order over each Dot’s write rows. It is always recomputed from the deltas, never obtained by subtraction, because floating-point addition is not invertible. **Reset** drops all Dots and restores V_0 , which is zero after a retrofit.

Removal of Dot k first computes its *taint closure* T . Process Dots after k in order, starting from a tainted slot set $\mathcal{T} = W_k$. A Dot j is tainted if $R_j \cap \mathcal{T} \neq \emptyset$, and a tainted Dot adds

Algorithm 1 Learning a Dot and recording it in the ledger

Require: frozen model with memory at layer ℓ ; ledger $\mathcal{L}$ with Dots $d_1..d_n$; examples E

- 1: $V \leftarrow \text{MATERIALIZE}(\mathcal{L}) \triangleright V = V_0 + \sum_j \Delta_j$ in Dot order
- 2: $h \leftarrow \text{layers}_{<\ell}(E)$; $(R, c) \leftarrow \text{reads of } E \text{ at layer } \ell$
- 3: $O \leftarrow \bigcup_j W_j \triangleright \text{slots owned by earlier Dots}$
- 4: $W \leftarrow \text{top-}B\{c_i \log \frac{D+1}{df_i+1} : i \in R \setminus O, m_i \leq q_{0.9}(m)\}$
- 5: $\theta \leftarrow V_W$; optimize θ with fresh Adam, seed $\text{hash}(E)$, deterministic kernels
- 6: append $d_{n+1} = (W, R, \Delta = \theta - V_W, \text{seed}, E)$ to $\mathcal{L}$;

return $\text{MATERIALIZE}(\mathcal{L})$

W_j to $\mathcal{T}$. Removal then drops d_k and walks the remaining Dots in order. Untainted Dots keep their deltas. Each tainted Dot is replayed from its logged data and seed on the state materialized from the Dots kept before it, recomputing its slot selection.

Proposition 1. Assume (i) the model and the address path are frozen; (ii) a Dot's training reads only the rows in its read set, and its optimizer state does not outlive it; (iii) slot selection depends only on the Dot's data, the frozen background statistics, and the owned slots inside its read set; and (iv) execution is deterministic. Then the state after removing Dot k by taint-closure replay is bitwise identical to the state obtained by learning the remaining Dots, in order, from V_0 .

Proof sketch. Induct over the Dots after k . For an untainted Dot j , every row in R_j has the same value in both histories, because rows in R_j were written either by untainted Dots, whose deltas are identical by induction, or by no Dot. Its owned set inside R_j is also identical, so by (ii)–(iv) its selection, gradients and delta are identical. For a tainted Dot, both histories replay the same data and seed from the same materialized state, so they agree by (iv). It remains to check that a replayed Dot cannot write a slot outside $\mathcal{T}$ that an untainted Dot reads. A replayed Dot's eligible set differs from the original only by slots that the removed or replayed Dots previously owned, and with a fixed budget its new top- B set lies inside its old write set together with those freed slots. All of these are in $\mathcal{T}$. $\square$

Fast removal drops Δ_k without replay. It is instant but not exact when later Dots read W_k . We measure both.

D. Context augmentation

A Dot trained only on the plain prompt `Question:`
`q\nAnswer:` is addressed through hidden states that depend on the whole context. When the same question is preceded by demonstrations, different slots are read and recall drops (§V). DOTS v2 therefore trains each fact in three contexts: the plain prompt and two demonstration prefixes (a two-shot and a three-shot prefix, listed in Appendix A). Evaluation uses a third demonstration prefix that never appears in training.

IV. EXPERIMENTAL SETUP

a) *Model and memory.* We retrofit Qwen2.5-0.5B [46] (24 layers, $d = 896$) in fp32 at layer $\ell = 15$, with $S = 512$

($N = 262,144$ slots), $k = 128$, $t = 32$ and $\tau = 1$. The value table holds 235M parameters (0.94 GB in fp32).

b) *Calibration data.* The background consists of 400 WikiText-2 training chunks of 128 tokens [47] and 1,500 NaturalQuestions-open training question-answer pairs [48], [49]. Of these pairs, 1,000 are in the plain format and 500 carry the evaluation demonstration prefix. Whitening statistics come from the first 12 mixed batches.

c) *Facts.* From TriviaQA [50] (rc.nocontext) we keep questions with answers of at most three words. The base model is evaluated with greedy decoding under a three-shot prefix. We take 1,000 training-split questions it answers *incorrectly* (base exact match on the candidates: 20.6%) as facts to learn. We take 300 validation-split questions it answers *correctly* as *known facts*, a retention probe. The facts to learn are split into ten Dots of 100, learned in sequence.

d) *Metrics.* We report: *taught EM*, exact match on all 1,000 learned facts in the training format; *unseen-context EM*, exact match on the first 200 learned facts under the held-out three-shot prefix; *known kept*, exact match on the 300 known facts under the same prefix (100% for the base model by construction); NaturalQuestions-open F1 on 300 validation questions; and perplexity on 12,800 WikiText-2 test tokens.

e) *DOTS settings.* Adam with learning rate 0.03 and batch size 16. v1 uses 120 steps and $B = 3,000$ write slots per Dot. v2 uses 240 steps and $B = 6,000$, because each Dot holds three contexts per fact. These settings were chosen in single-Dot sweeps as the configuration with the highest recall that kept at least 97% of a 100-fact known subset (Table III).

f) *Baselines.* Full fine-tuning updates all 358M non-embedding parameters with Adafactor [51]; the tied embedding matrix is frozen to fit memory. LoRA [28] uses rank 32 and $\alpha = 64$ on all attention and MLP projections (17.6M parameters) with AdamW. Both see the same ten chunks in the same order, with the same answer-only loss, batch size 16 and 60 steps per chunk. The variants marked +aug use the v2 context-augmented data and 120 steps. We sweep three learning rates for each baseline. All baseline runs except LoRA at 3×10^{-5} (77%) reach 98–100% recall on a chunk immediately after training on it (Table I), so their acquisition is not limited by the step budget.

g) *Hardware and reproducibility.* Main results come from a free Colab NVIDIA T4 GPU. The v1 pipeline was also run on a 6-core laptop CPU (Intel i7-9750H, 8 GB RAM). All runs are single executions. DOTS training is deterministic, and two independent T4 runs of v1 produced identical metrics.

V. RESULTS

a) *Learning without forgetting.* Table I and Figure 2 summarize the comparison. DOTS v1 learns 91.5% of the 1,000 facts while keeping 91.3% of the known facts. NQ-open F1 (10.3 against 9.9 for the base model) and perplexity (+1.1%) are essentially unchanged. The best full fine-tuning run keeps 51.3% of known facts, with NQ F1 falling to 6.9 and perplexity rising 15%. LoRA keeps at most 33.0%. Sparse writes also avoid interference among new facts. Figure 1 shows recall of each Dot after all ten were learned. Under DOTS

TABLE I
LEARNING 1,000 UNKNOWN TRIVIAQA FACTS AS TEN SEQUENTIAL DOTS ON QWEN2.5-0.5B (T4 GPU). “AFTER” IS RECALL OF THE FIRST CHUNK IMMEDIATELY AFTER IT WAS LEARNED. BASE MODEL: NQ EM 5.7, NQ F1 9.9, PERPLEXITY 27.43.

Method	Taught	Unseen ctx.	Known kept	NQ F1	PPL	After
DOTS v2 (augmented)	96.1	73.5	86.3	9.2	27.88	94
DOTS v1	91.5	10.5	91.3	10.3	27.75	95
Full FT, lr 5×10^{-6}	82.3	51.0	51.3	6.9	31.66	99
Full FT, lr 1×10^{-5}	83.7	52.0	39.3	6.1	35.65	100
Full FT, lr 2×10^{-5}	83.8	46.5	25.7	2.8	46.77	100
Full FT, lr 1×10^{-5} +aug	84.7	61.0	43.0	4.9	35.80	100
LoRA, lr 3×10^{-5}	49.2	16.5	27.7	5.4	39.28	77
LoRA, lr 1×10^{-4}	53.0	26.5	29.7	2.7	45.09	98
LoRA, lr 3×10^{-4}	37.4	3.5	6.3	0.0	134.57	100
LoRA, lr 1×10^{-4} +aug	57.9	22.0	33.0	4.8	47.72	100

the first Dot stays at 96%, while full fine-tuning and LoRA show a recency curve, with the first chunk at 62% and 28% respectively.

b) Context generalization.: DOTS v1 recalls taught facts in the training format (91.5%) but only 10.5% of them when the question follows an unseen three-shot prefix, below full fine-tuning (51.0%). The memory is addressed through context-dependent hidden states, and v1 trained only one context. Training each fact in two additional contexts (v2) raises unseen-context recall to 73.5%, above every baseline, including full fine-tuning on the same augmented data (61.0%). The cost is retention: known facts kept falls from 91.3% to 86.3%, because v2 writes twice as many rows.

A. Removal and reset

Table II reports removal of the first Dot. Exact removal produced a value table bitwise identical to a model that learned only Dots 2–10, in every configuration we ran: v1 and v2 on the T4 GPU and v1 on the laptop CPU. Recall of the removed facts falls to 6%. This equals the never-learned model, which answers 6% of them, presumably because some facts can be inferred from facts in other Dots. Dot 2’s recall is 97–98% after replay.

Fast removal is instant but damages later Dots: Dot 2 falls from 93% to 57%, because Dot 2 was learned while Dot 1’s rows were present. Reset restores a value table whose hash, and the model’s output logits, equal the original model’s.

a) Fan-out.: Every Dot read at least one slot written by every earlier Dot, so removing Dot k replays all later Dots. The fan-out was 100% in both versions. Removal cost is therefore linear in the number of later Dots: 194 s for nine Dots on the T4 for v1, and 3,247 s on the laptop CPU. This is the main open problem (§VI).

B. What makes writes safe

Table III isolates the components. Plain SGD barely moves the memory: the residual stream at layer 15 is large relative to the updated rows, and Adam’s normalized steps are needed. Background statistics drawn only from plain text leave the question-answer format’s own slots looking rare, so writes land

on slots that every question reads. Adding Q&A-format text to the background raises known-fact retention from 77% to 87%. Excluding the 10% of slots with the highest background read mass raises it to 98%; more steps then recover acquisition. In the earlier synthetic-memory prototype (Appendix A), ownership stopped new Dots from overwriting older ones. In the full ten-Dot runs here, earlier Dots also keep their recall (Figure 1).

a) Cost.: A v1 Dot of 100 facts took about 21 s on the T4 and 380 s on the laptop CPU. The ledger stored 10.3 MiB per Dot for v1 and 20.6 MiB for v2. Most of this is the logged deltas, which can be recomputed from the logged text when space matters.

VI. LIMITATIONS

- **Scale and scope.** One 0.5B model, one memory layer and one dataset of short factual answers. We do not show that memory writes can teach skills or longer documents. A 2026 retrofit study reported only small task gains from sparse memory finetuning [9].
- **Removal cost.** With 100% fan-out, exact removal replays every later Dot. Reducing fan-out, for example by restricting a Dot’s reads during training to a source-specific shard of slots, or by keeping periodic snapshots, is future work.
- **Retention trade-off.** Context augmentation improves generalization but lowers known-fact retention from 91.3% to 86.3%.
- **Baselines.** We swept three learning rates per baseline at a fixed step budget. Other schedules, replay, or regularization might change their retention. DOTS also used more steps per Dot (120 or 240) than the baselines (60 or 120).
- **Statistics.** All results are single runs. DOTS is deterministic given the data order, but we did not vary data order, fact sets, or the memory’s random seed.
- **Exactness boundary.** Proposition 1 covers knowledge written by Dots. Knowledge from pretraining is not removable by this mechanism, and facts the model can re-derive from remaining Dots are not leakage.

VII. BROADER IMPACT

A model that keeps learning on a user’s device learns whatever it is given, including false or harmful material.

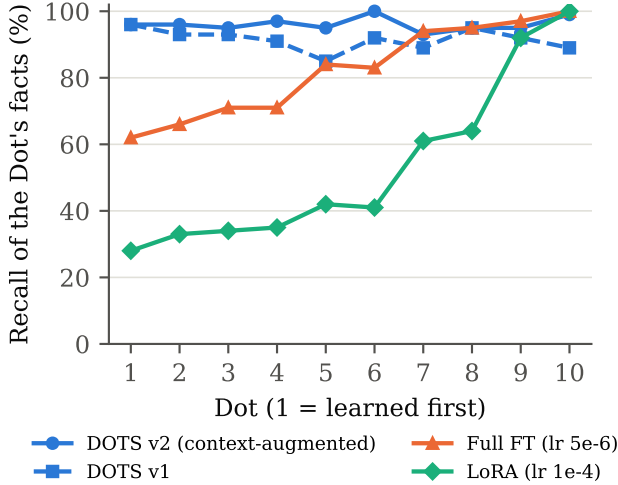


Fig. 1. Recall of each Dot's facts after all ten Dots were learned. Baselines forget earlier chunks; DOTS does not.

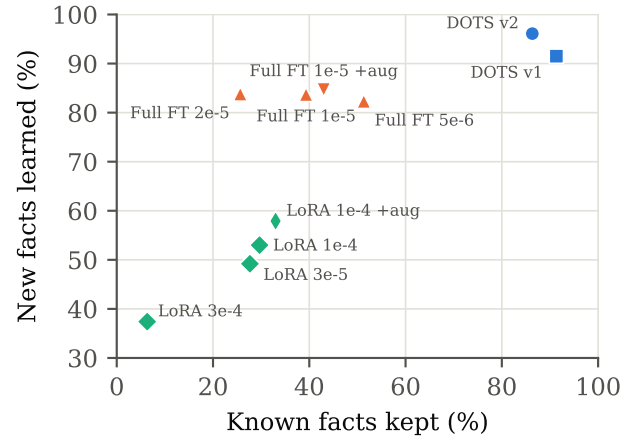


Fig. 2. New facts learned against known facts kept, for every run in Table I.

TABLE II
REMOVING DOT 1 AFTER ALL TEN DOTS WERE LEARNED (T4 GPU). “EQUAL TO NEVER LEARNED” COMPARES THE FULL VALUE TABLE BITWISE WITH A MODEL THAT LEARNED ONLY DOTS 2–10.

Version	Operation	Dot 1 EM	Dot 2 EM	Known kept	Time
v1	before removal	96	93	91.3	–
v1	fast (drop Δ_1)	3	57	93.7	<0.1 s
v1	exact (replay 9 Dots)	6	97	92.3	194 s
v2	fast (drop Δ_1)	4	58	90.3	<0.1 s
v2	exact (replay 9 Dots)	6	98	87.3	718 s

Exact removal equal to never learned: **yes** (v1 and v2, T4; v1 on CPU and in a 3-Dot smoke test)
Reset: value-table hash and output logits equal to the original model: **yes**

Small numbers of poisoned documents can backdoor models regardless of size [52]. A per-source ledger helps here: any source can be audited and exactly removed. It also supports data-deletion requests, which approximate unlearning cannot reliably meet [4]. Sharing Dots between users would make users distributors of what the Dots encode. Model-weight versioning systems such as Git-Theta [53] show how such sharing could be tracked.

VIII. CONCLUSION

DOTS retrofits a frozen language model with a sparse memory whose fixed hyperdimensional addresses, protected and owned write sets, and deterministic per-Dot training make continual factual learning both low-interference and exactly reversible. On Qwen2.5-0.5B it learns 96.1% of 1,000 new facts while keeping 86.3% of known facts, compared with at most 51.3% for tuned full fine-tuning. Removing any Dot yields a model bitwise identical to one that never learned it. The open problems are removal cost under full fan-out and evidence at larger scale and beyond facts.

CODE AND DATA AVAILABILITY

The code (retrofit, writer, ledger, evaluation and the self-contained Colab notebooks) and the exact fact lists will be

released at <https://github.com/sohan-a11y/open-dots>. All data come from public datasets: TriviaQA, NaturalQuestions-open and WikiText-2.

USE OF AI ASSISTANCE

An AI system (Claude, Anthropic) assisted with literature search, code, experiment design iterations and drafting of this manuscript. The author reviewed all content and takes full responsibility for it.

AUTHOR CONTRIBUTIONS, CONFLICTS OF INTEREST AND FUNDING

S.M.: conceptualization, investigation, writing (review and editing). The author declares no competing interests. This work received no external funding.

REFERENCES

- [1] M. McCloskey and N. J. Cohen, “Catastrophic interference in connectionist networks: The sequential learning problem,” in *Psychology of Learning and Motivation*, 1989, vol. 24, pp. 109–165.
- [2] J. Kirkpatrick, R. Pascanu, N. Rabinowitz *et al.*, “Overcoming catastrophic forgetting in neural networks,” *Proceedings of the National Academy of Sciences*, vol. 114, no. 13, pp. 3521–3526, 2017.
- [3] A. Ibrahim *et al.*, “Simple and scalable strategies to continually pre-train large language models,” *arXiv preprint arXiv:2403.08763*, 2024.

TABLE III
SINGLE-DOT ABLATIONS ON CPU (100 FACTS LEARNED; 100 KNOWN FACTS). ROWS ARE CUMULATIVE UNLESS NOTED.

Write policy	Learned	Known kept
SGD, lr 2–32, WikiText background	2–4	93–100
Adam, lr 0.03, WikiText background	92	77
+ Q&A-format background	90	87
+ read-mass protection (top 10%)	57	98
+ protection, lr 0.06	91	96
+ protection, 120 steps (selected)	95	99
anchor penalty instead of protection	58	95
protection + anchor penalty	38	100
memory at layer 19 (Adam 0.03, WikiText background)	56	79
budget $B = 1,000$ (Adam 0.03, Q&A background)	48	96

- [4] Z. Zhang, F. Wang, X. Li, Z. Wu, X. Tang, H. Liu, Q. He, W. Yin, and S. Wang, “Catastrophic failure of LLM unlearning via quantization,” in *International Conference on Learning Representations*, 2025, arXiv:2410.16454.
- [5] G. Lample, A. Sablayrolles, M. Ranzato, L. Denoyer, and H. Jégou, “Large memory layers with product keys,” in *Advances in Neural Information Processing Systems*, 2019, arXiv:1907.05242.
- [6] V.-P. Berges, B. Oğuz *et al.*, “Memory layers at scale,” *arXiv preprint arXiv:2412.09764*, 2024.
- [7] J. Lin, L. Zettlemoyer, G. Ghosh, W.-t. Yih, A. Markosyan, V.-P. Berges, and B. Oğuz, “Continual learning via sparse memory finetuning,” *arXiv preprint arXiv:2510.15103*, 2025.
- [8] S. Goyal, A. Kanchi, G. Shah, and P. Gupta, “Improving sparse memory finetuning,” *arXiv preprint arXiv:2604.05248*, 2026.
- [9] P. Gupta, G. Shah, S. Goyal, and A. Kanchi, “Sparse memory finetuning as a low-forgetting alternative to LoRA and full finetuning,” *arXiv preprint arXiv:2605.03229*, 2026.
- [10] L. Bourtole, V. Chandrasekaran, C. A. Choquette-Choo, H. Jia, A. Travers, B. Zhang, D. Lie, and N. Papernot, “Machine unlearning,” in *IEEE Symposium on Security and Privacy*, 2021, arXiv:1912.03817.
- [11] A. X, “Unlearning at scale: State-exact trace-preserving deletion in billion-parameter language models,” *arXiv preprint arXiv:2508.12220*, 2025.
- [12] K. Kuo, A. Setlur, K. Srinivas, A. Raghunathan, and V. Smith, “Exact unlearning of finetuning data via model merging at scale,” *arXiv preprint arXiv:2504.04626*, 2025.
- [13] H. Pouransari, D. Grangier, C. Thomas, M. Kirchhof, and O. Tuzel, “Pre-training with hierarchical memories: Separating long-tail and common knowledge,” in *International Conference on Learning Representations*, 2026, arXiv:2510.02375.
- [14] I. Schlag, K. Irie, and J. Schmidhuber, “Linear transformers are secretly fast weight programmers,” in *International Conference on Machine Learning*, 2021, arXiv:2102.11174.
- [15] S. Yang, J. Kautz, and A. Hatamizadeh, “Gated delta networks: Improving Mamba2 with delta rule,” in *International Conference on Learning Representations*, 2025, arXiv:2412.06464.
- [16] Y. Sun *et al.*, “Learning to (learn at test time): RNNs with expressive hidden states,” in *International Conference on Machine Learning*, 2025, arXiv:2407.04620.
- [17] A. Behrouz, P. Zhong, and V. Mirrokni, “Titans: Learning to memorize at test time,” *arXiv preprint arXiv:2501.00663*, 2025.
- [18] A. Behrouz, M. Razaviyayn, P. Zhong, and V. Mirrokni, “Nested learning: The illusion of deep learning architectures,” *arXiv preprint arXiv:2512.24695*, 2025.
- [19] Feng *et al.*, “In-place test-time training,” *arXiv preprint arXiv:2604.06169*, 2026.
- [20] T. Zhao and L. Jones, “Fast-weight product key memory,” *arXiv preprint arXiv:2601.00671*, 2026.
- [21] K. Meng, D. Bau, A. Andonian, and Y. Belinkov, “Locating and editing factual associations in GPT,” in *Advances in Neural Information Processing Systems*, 2022, arXiv:2202.05262.
- [22] K. Meng, A. S. Sharma, A. Andonian, Y. Belinkov, and D. Bau, “Mass-editing memory in a transformer,” in *International Conference on Learning Representations*, 2023, arXiv:2210.07229.
- [23] J. Fang *et al.*, “AlphaEdit: Null-space constrained knowledge editing for language models,” in *International Conference on Learning Representations*, 2025, arXiv:2410.02355.
- [24] T. Hartvigsen *et al.*, “Aging with GRACE: Lifelong model editing with discrete key-value adaptors,” in *Advances in Neural Information Processing Systems*, 2023, arXiv:2211.11031.
- [25] P. Wang *et al.*, “WISE: Rethinking the knowledge memory for lifelong model editing of large language models,” in *Advances in Neural Information Processing Systems*, 2024, arXiv:2405.14768.
- [26] P. Das *et al.*, “Larimar: Large language models with episodic memory control,” in *International Conference on Machine Learning*, 2024, arXiv:2403.11901.
- [27] Y. Wang *et al.*, “MEMORYLLM: Towards self-updatable large language models,” in *International Conference on Machine Learning*, 2024, arXiv:2402.04624.
- [28] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-rank adaptation of large language models,” in *International Conference on Learning Representations*, 2022, arXiv:2106.09685.
- [29] D. Biderman *et al.*, “LoRA learns less and forgets less,” *Transactions on Machine Learning Research*, 2024, arXiv:2405.09673.
- [30] S. Dohare *et al.*, “Loss of plasticity in deep continual learning,” *Nature*, vol. 632, pp. 768–774, 2024.
- [31] A. Rannen-Triki *et al.*, “Revisiting dynamic evaluation: Online adaptation for large language models,” *arXiv preprint arXiv:2403.01518*, 2024.
- [32] R. Zhang *et al.*, “Negative preference optimization: From catastrophic collapse to effective unlearning,” *arXiv preprint arXiv:2404.05868*, 2024.
- [33] A. Cloud *et al.*, “Gradient routing: Masking gradients to localize computation in neural networks,” *arXiv preprint arXiv:2410.04332*, 2024.
- [34] I. Shilov, A. Cloud, A. P. Gema, J. Goldman-Wetzler, N. Panickssery, H. Sleight, E. Jones, and C. Anil, “Beyond data filtering: Knowledge localization for capability removal in LLMs,” *arXiv preprint arXiv:2512.05648*, 2025.
- [35] W. Shi *et al.*, “FlexOlmo: Open language models for flexible data use,” *arXiv preprint arXiv:2507.07024*, 2025.
- [36] G. R. Ghosal, P. Maini, and A. Raghunathan, “Natively unlearnable large language models,” *arXiv preprint arXiv:2606.13873*, 2026.
- [37] T. A. Plate, “Holographic reduced representations,” *IEEE Transactions on Neural Networks*, vol. 6, no. 3, pp. 623–641, 1995.
- [38] P. Kanerva, “Hyperdimensional computing: An introduction to computing in distributed representation with high-dimensional random vectors,” *Cognitive Computation*, vol. 1, no. 2, pp. 139–159, 2009.
- [39] —, *Sparse Distributed Memory*. MIT Press, 1988.
- [40] T. Bricken and C. Pehevan, “Attention approximates sparse distributed memory,” in *Advances in Neural Information Processing Systems*, 2021, arXiv:2111.05498.
- [41] E. P. Frady, D. Kleyko, and F. T. Sommer, “A theory of sequence indexing and working memory in recurrent neural networks,” *Neural Computation*, vol. 30, no. 6, pp. 1449–1513, 2018.
- [42] M. S. Charikar, “Similarity estimation techniques from rounding algorithms,” in *Proceedings of the 34th Annual ACM Symposium on Theory of Computing*, 2002, pp. 380–388.
- [43] M. Heddes *et al.*, “Torchhd: An open source Python library to support research on hyperdimensional computing and vector symbolic architectures,” *Journal of Machine Learning Research*, vol. 24, 2023.
- [44] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *International Conference on Learning Representations*, 2015, arXiv:1412.6980.

- [45] PyTorch Contributors, “Reproducibility,” <https://pytorch.org/docs/stable/notes/randomness.html>, accessed 2026-10-02.
- [46] Qwen Team, “Qwen2.5 technical report,” *arXiv preprint arXiv:2412.15115*, 2024.
- [47] S. Merity, C. Xiong, J. Bradbury, and R. Socher, “Pointer sentinel mixture models,” in *International Conference on Learning Representations*, 2017, arXiv:1609.07843.
- [48] T. Kwiatkowski *et al.*, “Natural questions: A benchmark for question answering research,” *Transactions of the Association for Computational Linguistics*, vol. 7, 2019.
- [49] K. Lee, M.-W. Chang, and K. Toutanova, “Latent retrieval for weakly supervised open domain question answering,” in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019, arXiv:1906.00300.
- [50] M. Joshi, E. Choi, D. S. Weld, and L. Zettlemoyer, “TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension,” in *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics*, 2017, arXiv:1705.03551.
- [51] N. Shazeer and M. Stern, “Adafactor: Adaptive learning rates with sub-linear memory cost,” in *International Conference on Machine Learning*, 2018, arXiv:1804.04235.
- [52] A. Souly *et al.*, “Poisoning attacks on LLMs require a near-constant number of poison samples,” *arXiv preprint arXiv:2510.07192*, 2025.
- [53] N. Kandpal *et al.*, “Git-theta: A git extension for collaborative development of machine learning models,” in *International Conference on Machine Learning*, 2023, arXiv:2306.04529.

APPENDIX

Training and plain-format evaluation use `Question: {q}\nAnswer: with target {a}\n`; loss is on target tokens only. Training prefix A (two-shot): gold $\rightarrow$ Au; largest ocean $\rightarrow$ Pacific Ocean. Training prefix B (three-shot): inventor of the telephone $\rightarrow$ Alexander Graham Bell; tallest mountain $\rightarrow$ Mount Everest; legs of a spider $\rightarrow$ 8. Held-out evaluation prefix (three-shot, never trained): capital of Japan $\rightarrow$ Tokyo; Red Planet $\rightarrow$ Mars; painter of the Mona Lisa $\rightarrow$ Leonardo da Vinci.

Memory	layer 15; $S = 512$ ($N = 262,144$); $k = 128$; $t = 32$; $\tau = 1$; values zero-initialized
Calibration	400 $\times$ 128 WikiText-2 tokens + 1,500 NQ-open train Q&A; stats from 12 batches
DOTS v1	Adam 0.03; 120 steps; batch 16; $B = 3,000$; protection at the 90th percentile
DOTS v2	as v1 with 240 steps, $B = 6,000$, three contexts per fact
Full FT	Adafactor; lr $\{5 \times 10^{-6}, 10^{-5}, 2 \times 10^{-5}\}$; 60 steps per chunk; embeddings frozen
LoRA	$r = 32$, $\alpha = 64$, all projections; AdamW; lr $\{3 \times 10^{-5}, 10^{-4}, 3 \times 10^{-4}\}$; 60 steps
Determinism	deterministic PyTorch algorithms; cuBLAS workspace : 4096 : 8; eager attention on GPU

Before the retrofit, we tested the same mechanisms in a 2.6M-parameter byte-level model trained from scratch. It had an attention block, a gated delta-rule fast-weight block, and a 16,384-slot memory with fixed bipolar sub-keys. It was trained on 400 fictional entities and then learned two sets of 100 new entities. Full fine-tuning kept 12.4% of the original facts after two rounds. The DOTS write policy kept 93%, and 100% with a 65,536-slot memory. Slot ownership kept the first new set at 50% after the second was learned, against 28.5% without ownership. Exact removal and reset were bitwise exact.